

Types of Hash Functions in Data Structures

1. Division Method

- Formula: $h(k) = k \bmod m$, where k is the key and m is the size of the hash table.
- **Example:** For $k = 50$ and $m = 7$,
 $h(50) = 50 \bmod 7 = 1$.
The key 50 is mapped to index 1 in the hash table.

2. Multiplication Method

- Formula: $h(k) = \lfloor m(kA \bmod 1) \rfloor$, where A is a constant ($0 < A < 1$) and m is the table size.
- **Example:** For $k = 50$, $m = 10$, and $A = 0.618$,
 $h(50) = \lfloor 10(50 \times 0.618 \bmod 1) \rfloor = \lfloor 10 \times 0.9 \rfloor = 9$.
The key 50 is mapped to index 9.

3. Folding Method

- Splits the key into equal parts, adds them together, and applies modulo.
- **Example:** For $k = 123456$ and $m = 10$, split k into 123 and 456,
 $h(123456) = (123 + 456) \bmod 10 = 579 \bmod 10 = 9$.
The key is mapped to index 9.

4. Mid-Square Method

- Square the key and extract the middle digits.
- **Example:** For $k = 123$, $k^2 = 15129$.
If the middle two digits are 51, $h(123) = 51 \bmod m$. Assuming $m = 10$, $h(123) = 1$.

Indexing in Data Structures

Indexing refers to the process of organizing data such that specific elements can be efficiently accessed using their position or key. It is widely used in data structures to optimize retrieval and traversal of elements. Indexing is crucial for databases, arrays, linked lists, and other structures.

Types of Indexing

Single-Level Indexing

1. A single index is maintained to refer to the data items.

2. Example: Arrays.

Example:

python

Copy code

```
arr = [10, 20, 30, 40]print(arr[2]) # Output: 30 (access using index 2)
```

Multi-Level Indexing

1. A hierarchical indexing approach where an index points to other indexes.
2. Used in databases or large datasets where the size of the index exceeds memory.

Example: Multi-level indexes in a database can have:

1. Primary Index
 2. Secondary Index
-

Primary Index

1. The index is created based on a unique key (primary key) of the data.
2. Example: In relational databases, the primary index allows fast lookup of rows by their primary key.

Example:

Key	Record
101	A
102	B
103	C

Secondary Index

1. Provides an additional pathway to access data that is not based on the primary key.
2. Example: Searching based on attributes other than the primary key.

Example:Secondary Index Table

Age	Record
25	A
30	B
35	C

Clustered Indexing

1. Data is physically sorted in the same order as the index.
 2. Example: Pages in a dictionary sorted alphabetically.
-

Non-Clustered Indexing

1. The index stores pointers to the actual data.
2. Example: A book's index points to the page numbers.

Example in a Database:

Non-Clustered Index Table

Index	Pointer
A	Page 1
B	Page 5

Hash-Based Indexing

1. Uses hash functions to map keys to indexes.
2. Example: Hash tables for fast data retrieval.

Example:

```
hash_table = {1: 'A', 2: 'B', 3: 'C'}
print(hash_table[2]) # Output:
B
```

Sparse and Dense Indexing

1. **Sparse Indexing:** Only indexes some records, saving memory.
2. **Dense Indexing:** Maintains an index for every record.

Example Sparse Index:

Key	Pointer
1	Block 1
10	Block 10

Example Dense Index:

Key	Pointer
1	Block 1
2	Block 2
3	Block 3

1. Eulerian Path

An **Eulerian path** in a graph is a path that visits every edge exactly once. It may repeat vertices but cannot repeat edges.

Conditions for Existence:

1. For Undirected Graphs:

1. There are exactly **0 or 2 vertices** with an odd degree.

2. For Directed Graphs:

1. At most one vertex has $(\text{in-degree} = \text{out-degree} + 1)$ (start vertex).
2. At most one vertex has $(\text{out-degree} = \text{in-degree} + 1)$ (end vertex).
3. All other vertices must have equal in-degree and out-degree.

Special Case: Eulerian Circuit

If the path starts and ends at the same vertex, it's called an **Eulerian circuit**.

For an Eulerian circuit:

- **Undirected Graph:** All vertices have even degrees.
- **Directed Graph:** Every vertex has $\text{in-degree} = \text{out-degree}$.

Example (Undirected Graph):

Graph:

css

Copy code

A -- B -- C

| |

D -----E

Eulerian Path: $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow A$

Eulerian Circuit: Not possible (2 vertices have odd degrees).

2. Hamiltonian Path

A **Hamiltonian path** in a graph is a path that visits each vertex exactly once. Edges can be repeated.

Conditions for Existence:

There is no simple necessary and sufficient condition for a Hamiltonian path, but:

A **complete graph** always has a Hamiltonian path.

In a **connected graph**, finding such a path is computationally intensive (NP-complete problem).

Special Case: Hamiltonian Cycle

If the Hamiltonian path forms a closed loop (starts and ends at the same vertex), it is a **Hamiltonian cycle**.

Example:

Graph:

CSS

Copy code

A -- B -- C

| |

D -----E

Hamiltonian Path: $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D$

Hamiltonian Cycle: $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow A$

Key Differences

Aspect	Eulerian Path	Hamiltonian Path
Definition	Visits every edge exactly once.	Visits every vertex exactly once.

Edges vs. Vertices	Focuses on edges.	Focuses on vertices.
Conditions	Based on vertex degrees (odd/even).	No straightforward conditions.
Applications	Routing problems, circuit design.	Scheduling, traveling salesman problem.

shortest path

and Transitive Closure

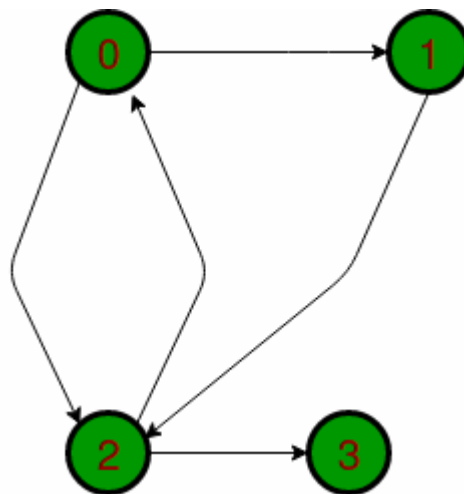
Single Source Shortest path-dijkstra algorithm

Link for example- <https://medium.com/basics/finding-the-shortest-path-with-a-little-help-from-dijkstra-613149fbdc8e>

All pair shortest path- Floyd Warshall

Transitive closure-Given a directed graph, determine if a vertex j is reachable from another vertex i for all vertex pairs (i, j) in the given graph. Here reachable means that there is a path from vertex i to j . The reach-ability matrix is called the transitive closure of a graph.

For example, consider below graph



Transitive closure of above graphs is

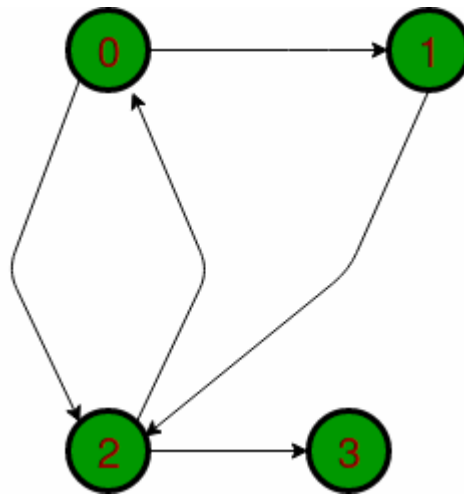
```

1 1 1 1
1 1 1 1
1 1 1 1
0 0 0 1
  
```

The graph is given in the form of adjacency matrix say 'graph[V][V]' where $\text{graph}[i][j]$ is 1 if there is an edge from vertex i to vertex j or i is equal to j , otherwise $\text{graph}[i][j]$ is 0.

Given a directed graph, determine if a vertex j is reachable from another vertex i for all vertex pairs (i, j) in the given graph. Here reachable means that there is a path from vertex i to j . The reach-ability matrix is called the transitive closure of a graph.

For example, consider below graph



Transitive closure of above graphs is

```
1 1 1 1
1 1 1 1
1 1 1 1
0 0 0 1
```

The graph is given in the form of adjacency matrix say 'graph[V][V]' where graph[i][j] is 1 if there is an edge from vertex i to vertex j or i is equal to j, otherwise graph[i][j] is 0.